OpenMP — Lesson 9

Locks

Gustavo Lozano

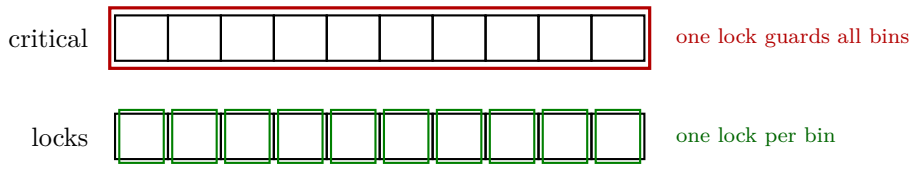
November 2026

Source: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the lock routines of the runtime library).

1 The idea

In Lesson 3, `critical` made an update safe by letting only one thread at a time into a block. But `critical` is **one lock for the whole block**: two threads that update *different* data still wait for each other.

Example: a histogram. Thread 0 adds to bin 17, thread 1 adds to bin 512. There is no conflict, yet with `critical` one of them waits. A **lock** is a variable you create yourself, so you can have **one lock per bin**. Threads then wait only when they really touch the same bin.



2 The lock routines

A lock has type `omp_lock_t` and a fixed life cycle:

Routine	Meaning
`omp_init_lock(&l)`	create the lock (unlocked); before any use
`omp_set_lock(&l)`	acquire it; if another thread holds it, **wait**
`omp_unset_lock(&l)`	release it; one waiting thread may now acquire it
`omp_destroy_lock(&l)`	free it; after the last use

The danger: if a thread forgets `omp_unset_lock` (for example, because of an early `return` or `break`), every other thread that wants that lock waits forever. The program **hangs** (a *deadlock*). `critical` cannot be forgotten this way, which is why it is preferred when one lock is enough.

3 The code

`locks.c`

```c
/*
 * locks.c -- OpenMP Lesson 9: locks.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (lock routines of the runtime library).
 *
 * Compile:
```

```c
 *     gcc -Wall -fopenmp -o locks locks.c
 * Run (use as many threads as you have cores, see `nproc`):
 *     OMP_NUM_THREADS=2 ./locks
 */

#include <stdio.h>
#include <omp.h>

#define N     20000000   /* values to put into the histogram          */
#define NBINS 1000       /* number of bins (try 2 in the experiment)  */

int hist[NBINS];
omp_lock_t lock[NBINS];  /* ONE lock per bin */

/* A cheap deterministic "random" bin for value i. */
static int bin_of(long i)
{
    return (int)((i * 2654435761UL) % NBINS);
}

int main(void)
{
    double t;
    long total;

    /* ===== Part 1: histogram protected by critical =====
       critical = ONE lock for the whole block. Two threads that update
       DIFFERENT bins still wait for each other.                          */
    for (int b = 0; b < NBINS; b++) hist[b] = 0;
    t = omp_get_wtime();
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        int b = bin_of(i);
        #pragma omp critical
        hist[b]++;
    }
    t = omp_get_wtime() - t;
    total = 0;
    for (int b = 0; b < NBINS; b++) total += hist[b];
    printf("critical      : %.3f s   total = %ld\n", t, total);

    /* ===== Part 2: histogram protected by one lock per bin =====
       Threads wait only if they update the SAME bin at the same time.   */
    for (int b = 0; b < NBINS; b++) {
        hist[b] = 0;
        omp_init_lock(&lock[b]);         /* 1. create (before use)       */
    }
    t = omp_get_wtime();
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        int b = bin_of(i);
        omp_set_lock(&lock[b]);          /* 2. acquire: wait if taken    */
        hist[b]++;
        omp_unset_lock(&lock[b]);        /* 3. release: ALWAYS do it     */
    }
    t = omp_get_wtime() - t;
    for (int b = 0; b < NBINS; b++)
        omp_destroy_lock(&lock[b]);      /* 4. free (after use)          */
    total = 0;
    for (int b = 0; b < NBINS; b++) total += hist[b];
    printf("lock per bin  : %.3f s   total = %ld\n", t, total);

    return 0;
}
```

```
gcc -Wall -fopenmp -o locks locks.c
OMP_NUM_THREADS=2 ./locks
```

Output on a 2-core machine (your times will differ):

```
critical      : 2.332 s   total = 20000000
lock per bin  : 0.481 s   total = 20000000
```

Key ideas

- Both histograms are correct (`total = N`). They differ only in how long threads wait.
- With one lock per bin, threads wait only when they hit the same bin at the same moment, which is rare with 1000 bins.
- Every `omp_set_lock` is matched by an `omp_unset_lock` on the same lock, a few lines later, with nothing in between that could skip it.

4 The experiment

Experiment

Question: when does a lock per data item beat one global lock?

1. Run with 1 thread and with `nproc` threads:

   ```
   OMP_NUM_THREADS=1 ./locks
   OMP_NUM_THREADS=$(nproc) ./locks
   ```

2. Change `#define NBINS 1000` to `#define NBINS 2`, recompile, and repeat step 1.

What to observe and answer.

1. With 1000 bins, why does `critical` get so much slower with more threads, while locks per bin suffer much less?
2. With 2 bins, why do locks per bin lose most of their advantage?
3. Even the lock version with several threads is slower than with 1 thread. Why, and what would be the real fix?

Answers (read after trying). (1) With `critical` every update needs the same lock, so all updates form a single line, and the lock is handed from core to core on almost every iteration (on the 2-core test machine: 0.26 s with 1 thread, 2.33 s with 2). With 1000 locks, two threads rarely want the same one at the same moment (0.48 s). (2) With 2 bins, half of the time both threads want the same bin, so they wait for each other again, just as with `critical` (2.06 s for locks vs. 3.19 s for `critical` on the test machine). A lock per item only helps when conflicts on the same item are rare. This is called *lock granularity*. (3) Each update still pays for acquiring and releasing a lock, and neighbouring bins and locks share cache lines, so the threads also suffer false sharing (Lesson 8). Locks make the update *safe*, not *fast*. The real fix is to avoid sharing: each thread fills a *private* histogram and they are added together once at the end. OpenMP can do this for you with an array reduction, `reduction(+:hist[:NBINS])`, available since OpenMP 4.5 (newer than the Chapman book; see van der Pas et al. [2]). On the test machine this version takes 0.010 s with 2 threads, against 0.48 s with locks.

Next lesson. Finding races with a tool: compiling with ThreadSanitizer and reading its report on the buggy counter from Lesson 2 (Chapman et al., Ch. 7).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.

[2] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.